Bài 2.4 – Logistic Regression

> **Giai đoạn 2 – Machine Learning cổ điển** · Thời lượng: 2 buổi · Hình thức mở đầu: *So sánh ví von*

🎯 Sau bài này bạn sẽ

- Hiểu vì sao không dùng Linear Regression cho bài toán phân loại.
- Hiểu hàm **sigmoid** biến mọi con số thành xác suất 0–1, tính tay được.
- Hiểu loss **Log Loss** (Binary Cross-Entropy): vì sao nó phạt nặng sự "tự tin sai".
- Xây mô hình **chấm điểm tín dụng** trên dữ liệu thật, đọc được xác suất và hệ số.

Chia buổi:

- **Buổi 1:** Mục 1 → 5 (dữ liệu, sigmoid, ngưỡng, log loss).
- **Buổi 2:** Mục 6 → 9 (huấn luyện, đọc hệ số, bài tập).

> Tên gọi gây hiểu nhầm: **Logistic Regression** có chữ "Regression" nhưng là thuật toán **phân loại**. Nó "hồi quy" ra một **xác suất**, rồi dùng xác suất đó để phân loại.

---

1. So sánh ví von: Cái công tắc đèn và núm vặn đèn

Hãy nghĩ về hai loại đèn:

```
Núm vặn đèn (dimmer)                Bảng điểm "khả năng vỡ nợ"
─────────────────────               ──────────────────────────
Vặn từ từ: tối → sáng               0% ──────────────────── 100%
Vặn quá tay vẫn chỉ sáng max        An toàn          Rủi ro cao
Không bao giờ "sáng âm"             Không bao giờ âm hay quá 100%
```

- **Linear Regression** giống một cái thước **dài vô tận**: cứ tăng là tăng mãi, có thể ra −30% hay 150%.
- **Logistic Regression** giống **núm vặn đèn**: dù bạn vặn mạnh đến đâu, độ sáng luôn nằm giữa "tắt" và "sáng tối đa". Đầu ra luôn là **xác suất từ 0 đến 1**.

Sau đó, ngân hàng đặt một **ngưỡng**: "khả năng vỡ nợ trên 50% thì từ chối". Đó là lúc núm vặn biến thành **công tắc** có/không.

---

2. Dữ liệu: chấm điểm tín dụng German Credit

2.1. Giới thiệu

Dữ liệu thật của 1.000 khách vay tại một ngân hàng Đức (thập niên 1990). Đơn vị tiền là **DM (Mác Đức)**. Mỗi khách được gắn nhãn `good` (trả nợ tốt) hoặc `bad` (vỡ nợ / rủi ro).

```
import numpy as np
import pandas as pd
```

```
from sklearn.datasets import fetch_openml

credit = fetch_openml("credit-g", version=1, as_frame=True).frame   # tải từ kho
OpenML
print(credit.shape)
print(credit[["checking_status", "duration", "credit_amount", "age",
"class"]].head())
```

Kết quả mẫu:

```
(1000, 21)
  checking_status  duration  credit_amount  age class
0              <0         6           1169   67  good
1        0<=X<200        48           5951   22   bad
2     no checking        12           2096   49  good
3              <0        42           7882   45  good
4              <0        24           4870   53   bad
```

Cột	Ý nghĩa
checking_status	Số dư tài khoản thanh toán (DM): `<0` (âm), `0<=X<200`, `>=200`, `no checking` (không có tài khoản)
duration	Thời hạn vay (tháng)
credit_amount	Số tiền vay (DM)
age	Tuổi
class	**Cột đích**: good / bad

(Còn 16 cột khác: mục đích vay, nghề nghiệp, tài sản... Bài 2.17 sẽ dùng hết.)

2.2. Tạo cột đích dạng số

```
credit["vo_no"] = (credit["class"] == "bad").astype(int)    # 1 = vỡ nợ, 0 = trả
tốt
print(credit["vo_no"].mean())                                # tỉ lệ vỡ nợ
print(credit.groupby("checking_status")["vo_no"].agg(["count", "mean"]).round(3))
```

Kết quả mẫu:

```
0.3
                count   mean
checking_status
0<=X<200          269  0.390
<0                274  0.493
```

```
>=200                63  0.222
no checking         394  0.117
```

- 30% khách vỡ nợ.
- Tài khoản **âm**: gần **một nửa** vỡ nợ (49%). Không có tài khoản thanh toán: chỉ 12%. (Có thể vì những người này có tài khoản tiết kiệm ở nơi khác.) Đặc trưng rất mạnh!

> **Quy ước:** trong bài toán rủi ro, ta thường gán **1 cho sự việc cần phát hiện** (vỡ nợ, gian lận, bệnh, spam). Gọi là **lớp dương (positive class)**.

---

3. Vì sao không dùng Linear Regression?

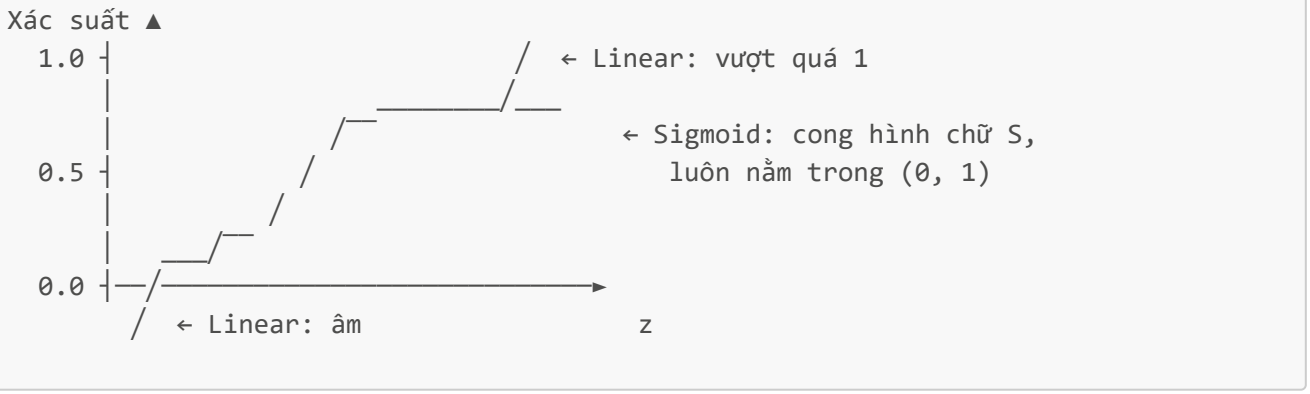
```
from sklearn.linear_model import LinearRegression

lr = LinearRegression().fit(credit[["duration"]], credit["vo_no"])
print(lr.predict([[4], [24], [72]]).round(3))    # "xác suất" vỡ nợ khi vay 4, 24, 72 tháng
```

Kết quả mẫu:

```
[0.162 0.325 0.718]
```

Trông có vẻ ổn. Nhưng đường thẳng kéo dài vô tận: với khoản vay 120 tháng, nó sẽ ra "xác suất" trên 1. Với khoản vay rất ngắn, có thể ra số âm. **Xác suất phải nằm trong [0, 1].** Ta cần "bẻ cong" đầu ra.



---

4. Hàm Sigmoid

4.1. Công thức

$$\sigma(z) = \frac{1}{}$$

```
            1 + e^(-z)
```

Ký hiệu	Ý nghĩa
z	"Điểm số" thô, tính bằng w·x + b (giống Linear Regression, bài 2.2). Có thể là bất kỳ số nào
e	Hằng số ≈ 2,718
σ(z) (đọc "xích-ma của z")	Xác suất, luôn nằm trong (0, 1)

Logistic Regression = Linear Regression + Sigmoid:

```
x ⟶ z = w·x + b ⟶ p = σ(z) ⟶ nếu p ≥ 0,5 → lớp 1, ngược lại → lớp 0
     (điểm thô)       (xác suất)    (quyết định)
```

4.2. Tính thử

- z = 0: $e^0 = 1$ → σ = 1 / (1 + 1) = **0,5**
- z = 2: $e^{-2} \approx 0{,}135$ → σ = 1 / 1,135 ≈ **0,881**
- z = −2: $e^2 \approx 7{,}389$ → σ = 1 / 8,389 ≈ **0,119**

```python
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

for z in [-4, -2, 0, 1, 2, 4]:
    print(z, round(sigmoid(z), 3))
```

Kết quả mẫu:

```
-4 0.018
-2 0.119
0 0.5
1 0.731
2 0.881
4 0.982
```

Ba tính chất cần nhớ:

- z = 0 → xác suất 50%: "lưỡng lự".
- z dương lớn → gần 1. z âm lớn → gần 0.
- Đối xứng: σ(−z) = 1 − σ(z). (0,119 + 0,881 = 1.)

4.3. Ý nghĩa của z: log-odds

- **Odds (Tỉ số cược):** p / (1 − p). Ví dụ p = 0,8 → odds = 0,8 / 0,2 = 4, nghĩa là "khả năng xảy ra gấp 4 lần khả năng không xảy ra". Dân cá cược hay nói "4 ăn 1".
- Có thể chứng minh: **z = log(odds)**. Nên mỗi hệ số w cho biết: đặc trưng tăng 1 đơn vị thì **log-odds** tăng w, tức **odds nhân lên e^w lần**. Ta dùng điều này ở mục 7.

---

5. Hàm loss: Log Loss (Binary Cross-Entropy)

5.1. Vì sao không dùng MSE?

Với sigmoid, MSE tạo ra một "thung lũng" gồ ghề nhiều hố, Gradient Descent dễ kẹt. Và MSE phạt quá nhẹ khi mô hình **tự tin mà sai**.

5.2. Công thức

```
Với một mẫu:  Loss = -[ y·log(p) + (1 - y)·log(1 - p) ]
```

Ký hiệu	Ý nghĩa
y	Nhãn thật (0 hoặc 1)
p	Xác suất mô hình dự đoán cho lớp 1
log	Logarit tự nhiên

Vì y chỉ là 0 hoặc 1, công thức thực ra chỉ còn **một vế**:

- Nếu y = 1: Loss = −log(p). Muốn loss nhỏ thì p phải gần 1.
- Nếu y = 0: Loss = −log(1 − p). Muốn loss nhỏ thì p phải gần 0.

5.3. Tính thử: khách thật sự vỡ nợ (y = 1)

```python
for p in [0.9, 0.6, 0.1]:              # mô hình đoán xác suất vỡ nợ là p
    print(p, round(-np.log(p), 3))     # loss = -log(p)
```

Kết quả mẫu:

```
0.9 0.105
0.6 0.511
0.1 2.303
```

Mô hình nói	Loss	Nhận xét
"90% vỡ nợ"	0,105	Đúng và tự tin → phạt rất nhẹ
"60% vỡ nợ"	0,511	Đúng nhưng lưỡng lự → phạt vừa

Mô hình nói	Loss	Nhận xét
"10% vỡ nợ"	**2,303**	**Tự tin mà sai** → phạt nặng gấp 22 lần

Nếu p → 0, −log(p) → vô cùng. Log Loss **không tha thứ** cho sự tự tin sai lầm. Đây cũng là loss của mọi mạng nơ-ron phân loại (sẽ gặp lại ở bài 3.5).

---

6. Huấn luyện mô hình chấm điểm tín dụng

6.1. Chuẩn bị và chia dữ liệu

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

X = pd.get_dummies(credit[["duration", "credit_amount", "age",
"checking_status"]],
                   columns=["checking_status"], drop_first=True, dtype=int)   #
one-hot (bài 2.2)
print(X.columns.tolist())
y = credit["vo_no"]

# stratify=y: giữ tỉ lệ 30% vỡ nợ ở cả train và test (bài 1.11)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, random_state=42,
stratify=y)
sc = StandardScaler().fit(Xtr)            # Logistic Regression học bằng gradient
→ nên chuẩn hóa
Xtr_s, Xte_s = sc.transform(Xtr), sc.transform(Xte)
```

Kết quả mẫu:

```
['duration', 'credit_amount', 'age', 'checking_status_<0',
'checking_status_>=200', 'checking_status_no checking']
```

Nhóm gốc của `checking_status` là `0<=X<200` (bị `drop_first` bỏ cột).

6.2. Huấn luyện và dự đoán

```
mh = LogisticRegression()
mh.fit(Xtr_s, ytr)

print(mh.predict_proba(Xte_s[:5]).round(3))    # xác suất [lớp 0, lớp 1] cho 5 khách đầu
print(mh.predict(Xte_s[:5]), yte.values[:5])   # quyết định (ngưỡng 0.5) vs nhãn thật
print(round(mh.score(Xte_s, yte), 3))          # accuracy: tỉ lệ đoán đúng
```

Kết quả mẫu:

```
[[0.66  0.34 ]
 [0.699 0.301]
 [0.536 0.464]
 [0.537 0.463]
 [0.558 0.442]]
[0 0 0 0 0] [0 0 1 0 1]
0.74
```

Đọc kết quả:

- `predict_proba` trả về 2 cột: cột 0 là P(trả tốt), cột 1 là P(vỡ nợ). Mỗi hàng cộng lại bằng 1.
- Khách thứ 3 và thứ 5 thật sự vỡ nợ. Mô hình cho họ xác suất 46% và 44%: **cao hơn** mức trung bình 30%, nhưng **chưa vượt ngưỡng 50%**, nên `predict` vẫn ra 0. Ngưỡng 0,5 có phải lựa chọn đúng không? Bài 2.5 sẽ trả lời.
- Accuracy 74%. Nghe khá ổn? Nhưng nhớ: đoán **tất cả** "trả tốt" cũng đúng 70%. Bài 2.5 sẽ dạy cách đánh giá cẩn thận hơn.

6.3. Tự tính lại một dự đoán

```
z = Xte_s[0] @ mh.coef_[0] + mh.intercept_[0]    # z = w·x + b (tích vô hướng, bài 1.8)
print(round(z, 3), round(sigmoid(z), 3))         # sigmoid(z) phải khớp predict_proba
```

Kết quả mẫu:

```
-0.661 0.34
```

Khớp với 0,34 ở hàng đầu của `predict_proba`. Không có phép màu nào: chỉ là **tích vô hướng + sigmoid**.

6.4. Log loss so với mốc

```
from sklearn.metrics import log_loss
print(round(log_loss(yte, mh.predict_proba(Xte_s)[:, 1]), 3),        # mô hình
      round(log_loss(yte, np.full(len(yte), ytr.mean())), 3))        # mốc: ai
cũng 30%
```

Kết quả mẫu:

```
0.491 0.611
```

Mô hình có loss thấp hơn mốc "cho ai cũng 30%" → xác suất của mô hình **có thông tin thật**.

---

7. Đọc hệ số

```
coef = pd.Series(mh.coef_[0], index=X.columns)
print(coef.round(3).sort_values())
print(np.exp(coef).round(2).sort_values())      # e^w: odds nhân lên bao nhiêu lần
print(round(mh.intercept_[0], 3))
```

Kết quả mẫu:

```
checking_status_no checking   -0.712
age                           -0.104
checking_status_>=200         -0.102
credit_amount                  0.044
checking_status_<0             0.199
duration                       0.388
dtype: float64
checking_status_no checking    0.49
age                            0.90
checking_status_>=200          0.90
credit_amount                  1.05
checking_status_<0             1.22
duration                       1.47
dtype: float64
-1.023
```

Vì đã chuẩn hóa, mỗi hệ số tương ứng với "đặc trưng tăng **1 độ lệch chuẩn**":

Đặc trưng	Hệ số	e^w	Đọc là
duration	+0,388	1,47	Vay dài hơn 1 ĐLC (khoảng 12 tháng) → **odds vỡ nợ ×1,47**
checking_status_<0	+0,199	1,22	Tài khoản âm → rủi ro cao hơn nhóm gốc

Đặc trưng	Hệ số	e^w	Đọc là
age	−0,104	0,90	Lớn tuổi hơn → rủi ro thấp hơn một chút
checking_status_no checking	−0,712	0,49	Không có tài khoản thanh toán → odds vỡ nợ **giảm khoảng một nửa**

- Hệ số **dương** → tăng rủi ro vỡ nợ. **Âm** → giảm rủi ro.
- Hệ số của các cột one-hot sau khi chuẩn hóa khó đọc chính xác về độ lớn, nên chỉ cần đọc **dấu** và so sánh tương đối.

> 💡 Ngân hàng rất thích Logistic Regression vì **giải thích được**: "Hồ sơ bị từ chối chủ yếu do thời hạn vay dài và tài khoản đang âm." Ở nhiều nước, luật yêu cầu ngân hàng phải giải thích được lý do từ chối cho vay.

---

8. Nhiều hơn 2 lớp

Logistic Regression trong Scikit-learn tự xử lý bài toán **nhiều lớp** (ví dụ phân loại email: công việc / cá nhân / quảng cáo / spam). Nó dùng hàm **softmax**, phiên bản nhiều lớp của sigmoid (sẽ học kỹ ở bài 3.2). Code không đổi: `predict_proba` khi đó trả về nhiều cột, mỗi lớp một cột.

---

9. Bài tập

1. **Tính tay** σ(1,5) và σ(−0,5). Kiểm tra bằng hàm `sigmoid`.
2. Khách có p(vỡ nợ) = 0,2. Odds là bao nhiêu? Nếu một đặc trưng có e^w = 2 và khách tăng đặc trưng đó 1 đơn vị, odds mới và xác suất mới là bao nhiêu?
3. Dùng mô hình ở mục 6, dự đoán xác suất vỡ nợ cho hai khách:
   - Khách A: vay 36 tháng, 8.000 DM, 25 tuổi, tài khoản **âm**.
   - Khách B: vay 12 tháng, 2.000 DM, 45 tuổi, **không có** tài khoản thanh toán.

   (Gợi ý: tạo DataFrame có **đúng các cột** như `X.columns`, rồi `sc.transform` trước khi `predict_proba`.)
4. Thêm cột `savings_status` (tình trạng tiết kiệm) vào mô hình (nhớ one-hot). Log loss trên test có giảm không?

Đáp án

> Mẹo Notion: chọn tiêu đề "Đáp án" → `Turn into → Toggle heading 3`.

1. σ(1,5) = 1 / (1 + $e^{-1,5}$) = 1 / (1 + 0,2231) ≈ **0,818**. σ(−0,5) = 1 / (1 + $e^{0,5}$) = 1 / (1 + 1,6487) ≈ **0,378**.
2. Odds = 0,2 / 0,8 = **0,25**. Nhân 2 → odds mới = **0,5**. p mới = odds / (1 + odds) = 0,5 / 1,5 ≈ **0,333**. Để ý: odds gấp đôi **không** có nghĩa xác suất gấp đôi (0,2 → 0,333, không phải 0,4).
3. Code:

```
kh = pd.DataFrame([
    {"duration": 36, "credit_amount": 8000, "age": 25,
```

```
     "checking_status_<0": 1, "checking_status_>=200": 0, "checking_status_no
checking": 0},
    {"duration": 12, "credit_amount": 2000, "age": 45,
     "checking_status_<0": 0, "checking_status_>=200": 0, "checking_status_no
checking": 1},
])[X.columns]                                  # sắp đúng thứ tự cột như lúc huấn
luyện
print(mh.predict_proba(sc.transform(kh)).round(3))
# [[0.359 0.641]    ← Khách A: 64% vỡ nợ → rủi ro cao
#  [0.917 0.083]]   ← Khách B: 8% vỡ nợ → rủi ro thấp
```

4. Code (gợi ý):

```
X2 = pd.get_dummies(credit[["duration", "credit_amount", "age", "checking_status",
"savings_status"]],
                    columns=["checking_status", "savings_status"],
drop_first=True, dtype=int)
# chia lại với cùng random_state và stratify, chuẩn hóa, fit, rồi tính log_loss
như mục 6.4
```

Kết quả thật: log loss **tăng nhẹ** từ 0,491 lên 0,494, tức là **không tốt hơn**. Bất ngờ? Đặc trưng nghe có vẻ hữu ích chưa chắc giúp mô hình: có thể thông tin tiết kiệm đã trùng với `checking_status`, và thêm 4 cột one-hot với chỉ 800 mẫu train làm mô hình "nhiễu" hơn. Bài học: **luôn kiểm chứng bằng số liệu, đừng tin trực giác** (sẽ học cách kiểm chứng chắc chắn hơn bằng cross-validation ở bài 2.17).

---

10. Flashcard

Mặt trước	Mặt sau
Logistic Regression là hồi quy hay phân loại?	Phân loại (hồi quy ra xác suất, rồi dùng ngưỡng).
Công thức sigmoid?	σ(z) = 1 / (1 + e^(−z))
σ(0) bằng bao nhiêu?	0,5
Logistic Regression = ?	z = w·x + b, rồi p = σ(z).
Odds là gì?	p / (1 − p)
e^w của một hệ số nghĩa là?	Đặc trưng tăng 1 đơn vị → odds nhân e^w lần.
Loss của Logistic Regression?	Log Loss: −[y·log(p) + (1−y)·log(1−p)]
Vì sao Log Loss tốt?	Phạt rất nặng khi mô hình tự tin mà sai.
`predict_proba` trả về gì?	Xác suất từng lớp, mỗi hàng cộng lại bằng 1.
Lớp dương (positive) thường là?	Sự việc cần phát hiện: vỡ nợ, gian lận, bệnh, spam.

Bài tiếp theo: 2.5 – Đánh giá phân loại: Confusion Matrix, Accuracy, Precision, Recall, F1, ROC-AUC.